Modelos de ondas em GPUs

Portes do WAVEWATCH III em C++/Kokkos e em Rust que precisam dar os bits do Fortran

Matheus Hoffmann ORCID 0009-0009-1056-7661
Escola Politécnica, Universidade Federal do Rio de Janeiro, Brasil

Pedro Veras Guimarães ORCID 0000-0002-1686-4722
Coorientador do primeiro autor. LabECO, Departamento de Engenharia Mecânica, Universidade Federal de Santa Catarina, Brasil

Pedro Cruz ORCID 0000-0002-2019-133X
Orientador do primeiro autor. Grupo de Teleinformática e Automação (GTA), DEL, Escola Politécnica, Universidade Federal do Rio de Janeiro, Brasil

2026-11-11 · v0 (preprint, sem revisão por pares) · rev. e7996b3+274

Resumo. Este texto relata dois portes do modelo espectral de ondas WAVEWATCH III (WW3). Agentes de programação escreveram os dois portes, e os dois portes têm uma regra de aceitação só: o porte precisa dar os mesmos bits do Fortran. Uma tolerância é um diagnóstico e nunca é o veredito. O *ww3-gpu* mantém o WW3, e o Fortran chama código C++ escrito com Kokkos, atrás de uma flag de execução para cada rotina. A referência do ww3-gpu é o caminho Fortran do mesmo executável. O *chaceon* substitui o WW3 por Rust, compilado para a CPU, para SPIR-V (rust-gpu) e para PTX (Rust-CUDA). Nenhum Fortran roda no processo do chaceon, e a referência do chaceon é um WW3 compilado em separado. O ww3-gpu porta a aproximação de interação discreta (DIA) da transferência entre quatro ondas. Na CPU, a DIA portada é idêntica byte a byte nas 100 entradas seriais e nas 25 entradas OpenMP da matriz de regressão do WW3 que selecionam a DIA. A DIA ocupa 48,4 % do tempo próprio em uma thread, num caso de 9.520 pontos. O chaceon roda `ww3_grid`, `ww3_strt`, `ww3_prep` e `ww3_shel` sem Fortran, e 96 arquivos de saída são idênticos byte a byte à referência. Nenhum dos dois portes rodou o modelo inteiro numa GPU real, e este texto não afirma ganho de velocidade. O texto compara como cada projeto define e confere a própria evidência.

Palavras-chave: WAVEWATCH III; reprodutibilidade bit a bit; Kokkos; Rust; porte para GPU; agentes de programação.

Prefácio

Este projeto existe para apoiar o Ondas & Ventos (Universidade Federal de Santa Catarina 2026), um sistema operacional de previsão de ondas da Universidade Federal de Santa Catarina (UFSC). O LabECO da UFSC roda o WW3 de forma operacional para a ReNOMO, a Rede Nacional de Observação e Monitoramento Oceânico (LabECO/UFSC 2025). A ReNOMO começou em 2023, com a chamada CNPq/MCTI/Finep 062/2022. Mais de 100 pesquisadores de 31 instituições brasileiras participam da rede (LabECO/UFSC 2025).

O sistema precisa rodar o modelo de ondas a cada ciclo de previsão, e o tempo de execução do WW3 limita o ciclo. O sistema também planeja forçar o modelo de ondas com mais de um modelo de vento: o IFS do ECMWF, o GFS do NCEP e o WeatherNext do Google ⚠. Cada modelo de vento acrescenta uma execução completa do WW3, e por isso um conjunto de modelos de vento multiplica o custo do ciclo. Um WW3 mais rápido em GPU, com os mesmos bits, torna essas execuções viáveis sem mudança na previsão. O ww3-gpu e o chaceon trabalham para chegar a esse WW3, e este texto não afirma ganho de velocidade.

0 Contexto: um modelo oceânico portado por um modelo de linguagem

Koldunov et al. (Koldunov et al. 2026) relatam o porte de um modelo oceânico inteiro por um modelo de linguagem grande. O modelo é o FESOM2, um modelo de oceano e gelo marinho, e os autores trabalham no Alfred Wegener Institute. O Claude Code, com o Claude Opus 4.7, traduziu o FESOM2 de Fortran para C e depois de C para C++/Kokkos. As pessoas definiram a estratégia, revisaram os planos e os diffs e encontraram erros de física e de validação. O agente leu, traduziu, construiu os ambientes de teste, depurou, mediu o desempenho e escreveu a documentação.

A etapa em C é o centro do método de Koldunov et al. O código C limpo é a referência, e cada kernel Kokkos mantém o original em C como "gêmeo". A regra de aceitação muda a cada etapa. Os autores comparam o modelo

em C com o Fortran por estatística: em cinco anos, a diferença RMS da média é 0,006 °C na temperatura da superfície do mar. Só o modelo Kokkos Serial é idêntico byte a byte, e só ao modelo em C. Os autores escrevem que a concordância bit a bit entre Fortran e C "across compilers and arithmetic orderings is neither attainable nor necessary". Na GPU, os autores aceitam diferenças até um teto registrado para cada campo.

O sucessor do WW3 também está em desenvolvimento. A NCEP Office Note 525 (Tolman 2025) escolheu uma reescrita de baixo para cima, com um núcleo em C++ primeiro e um núcleo em Rust depois ⚠. O mesmo relatório rebaixa o Fortran a uma linguagem só de solvers ⚠. O repositório do WAVEWATCH IV (NOAA Environmental Modeling Center 2026) está em pré-alfa: em 2026-11-11, o branch `develop` tinha 40 commits, de 2025-11-17 a 2026-10-16 (`875f49a`). Os 97 arquivos-fonte do WAVEWATCH IV são C++ (66 `.cpp`, 31 `.h`), e nenhum kernel de física do WW3 está no repositório ainda.

Por isso, os kernels centrais do modelo de ondas continuam em Fortran por alguns anos ⚠. Esse é o primeiro motivo para o primeiro autor continuar a manter o ww3-gpu e o chaceon. Os dois projetos fazem os kernels Fortran do WW3 rodarem em GPU agora, sem mudança nos bits.

Este texto parte desse resultado e faz uma pergunta mais estreita. Um agente consegue portar um modelo espectral de ondas e dar os bits do próprio Fortran? Os dois portes abaixo não têm etapa em C, e a referência dos dois portes é o Fortran. Os dois portes também usam duas práticas que o FESOM2 relata: tradução estritamente literal e, para cada afirmação sobre o Fortran, um `file:line` com o valor literal lido ali.

1 Por que bit a bit

O WW3 integra a equação de balanço de ação

$$\frac{\partial N}{\partial t} + \nabla \cdot (\dot{\mathbf{c}}\, N) = \frac{S}{\sigma},$$

em que o termo-fonte S é a soma da entrada de vento, da dissipação, da transferência não linear e dos termos de fundo. Um porte que muda a ordem de uma soma em ponto flutuante muda N. Um sistema de previsão validado contra observações precisa então ser validado de novo. Por isso, os dois projetos mantêm a aritmética do Fortran. Os dois projetos não reordenam reduções, não usam multiplicação-adição fundida (`-ffp-contract=off` nos dois lados) e não alargam tolerâncias. Uma diferença é defeito do porte até duas execuções do próprio Fortran darem bytes diferentes no mesmo arquivo.

2 Duas formas para uma regra

Tabela 1: Os dois portes. Fontes: chaceon `docs/WW3_GPU_PARITY_202611.md` (2026-11-09) e `README.md` em `0c098fe` (Hoffmann 2026a); ww3-gpu `kokkos/PORT_STATUS.md` em `58167d7` (Hoffmann 2026c).

	ww3-gpu (C++/Kokkos)	chaceon (Rust)
Forma	Dentro do WW3: o Fortran chama C++ por um shim `bind(C)`, uma flag para cada rotina	Substitui o WW3: nenhum Fortran num processo do chaceon
Referência	NOAA-EMC/WW3 `03adda91`, por um fork que acrescenta 5.687 linhas e não remove nenhuma	MetOcean WW3 7.14, `92e2cf66`
Paridade na CPU contra	o caminho Fortran do mesmo executável	uma referência compilada em separado
Alvos de dispositivo	Kokkos 5.2.2, backend CUDA (`sm_89`); OpenMP na CPU	rust-gpu (SPIR-V, Vulkan) e Rust-CUDA (PTX)
Espectro residente no dispositivo	sim, no backend OpenMP	sim, em SPIR-V no Vulkan de CPU (lavapipe)
Tamanho do porte	62.257 linhas de C++, 355 arquivos (2026-10-14)	29.229 linhas de porte em Rust, 5.936 linhas de ferramentas
Métodos formais	consultas SMT, varreduras exaustivas	uma prova em Lean 4 sobre a divisão de `f32`
Licença	MIT, kernels em LGPL-3.0	MPL-2.0

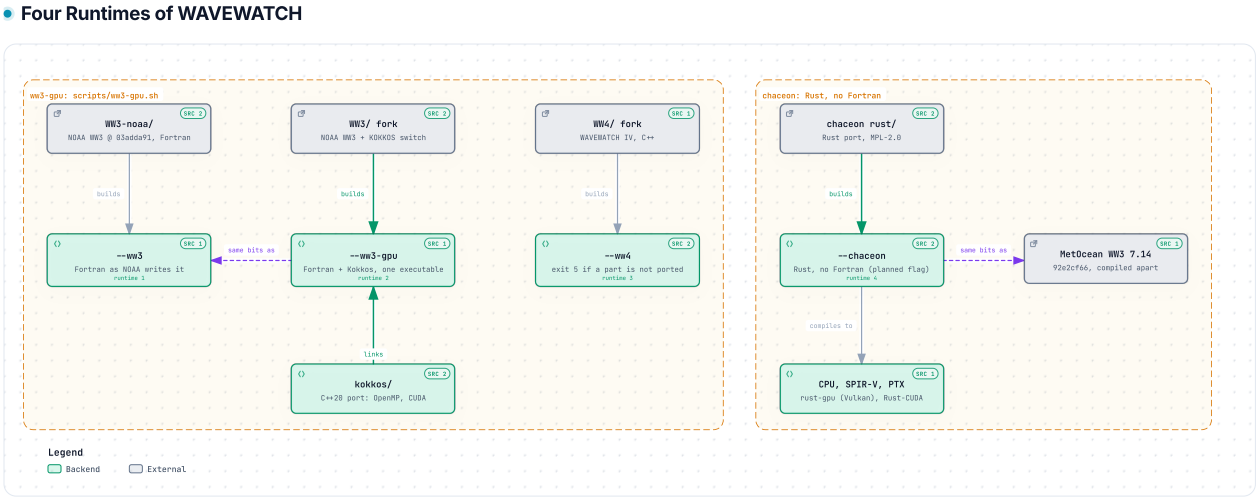


Figura 1: Os quatro runtimes (desenho em inglês). Uma seta contínua mostra o que compila ou liga o quê; uma seta tracejada aponta para a execução que precisa dar os mesmos bits. Desenho do Archify a partir de `docs/archify/ww3-gpu-four-runtimes.json` (Hoffmann 2026c).

Tabela 2: Os dois repositórios. Fonte: a API REST do GitHub (`repos/h0ffmann/<repo>` para `created_at` e `pushed_at`, `commits?sha=main` para os commits), lida em 2026-11-11 às 03h35 UTC. O primeiro commit do ww3-gpu é mais antigo que o repositório porque o commit foi feito antes do primeiro push.

Métrica	ww3-gpu	chaceon
Repositório criado	2026-09-12, 01h31 UTC	2026-10-23, 03h58 UTC
Primeiro commit no `main`	2026-09-11, 13h47 UTC, `b43a6d3`	2026-10-23, 03h58 UTC, `3bad31a`
Último commit no `main`	2026-11-11, 03h16 UTC, `efc0211`	2026-11-09, 08h53 UTC, `0c098fe`
Último push, qualquer branch	2026-11-11, 03h31 UTC	2026-11-09, 08h53 UTC
Commits no `main`	800	110

A rota Kokkos mantém todos os programas do WW3 e substitui uma rotina de cada vez. Por isso, um regtest consegue isolar o efeito de uma rotina. A rota Rust precisa reconstruir os programas, os formatos de arquivo e o laço de tempo antes de comparar uma execução inteira. Depois disso, nenhum Fortran sobra para dividir estado com o porte.

O registro de progresso do ww3-gpu para a NOAA (Hoffmann 2026e) guarda os resultados de CPU do runtime ww3-gpu inteiro, com o espectro residente, e os limites desses resultados. O porte da DIA também está publicado sozinho, como ww3-dia-gpu-v2 (Hoffmann 2026d). O repositório ww3-dia-gpu-v2 compara o porte com um kernel CUDA anterior da mesma rotina.

A Figura 1 mostra os quatro runtimes. O ww3-gpu tem três, todos atrás de um script só, `scripts/ww3-gpu.sh`. O `--ww3` compila o WW3 puro da NOAA a partir de `WW3-noaa/` e é a referência. O `--ww3-gpu` compila o fork com a switch KOKKOS e liga o porte Kokkos. Nesse executável, uma flag para cada rotina escolhe o código Fortran ou o código Kokkos. O `--ww4` compila o WAVEWATCH IV a partir do fork do WAVEWATCH IV e para com o código de saída 5 quando o caso precisa de uma parte não portada. O chaceon é o quarto runtime, `--chaceon`, uma flag que o script ainda não tem ⚠. O chaceon roda o WW3 reescrito em Rust, sem Fortran no processo. A referência do chaceon é o WW3 7.14 da MetOcean, compilado em separado. Hoje, o ww3-gpu fixa o WW3 da NOAA em `03adda91`, e o chaceon fixa o WW3 7.14.

3 Escadas de evidência

Cada projeto dá nome aos próprios degraus. Um degrau num registro não é a mesma evidência que um degrau no outro registro.

Tabela 3: Degraus de evidência. As contagens do ww3-gpu vêm de `scripts/port_score.py --write` (perfil `bench_small`, 120 × 80 pontos, 12 h, uma thread, gprof, 2026-10-14); as contagens do chaceon vêm de `just port-summary`.

Degrau	ww3-gpu	chaceon
O código existe	*translated*: 316 rotinas, 26,2 % das linhas de Fortran	*translated*: o Rust existe, sem cobertura completa
Uma execução chama o código	*ported*: 72 rotinas, 99,3 % do tempo de execução medido	*routine*: toda chamada capturada é idêntica bit a bit no host e em SPIR-V
Execuções inteiras idênticas, CPU	*regtested*: 14 rotinas, 48,6 % do tempo de execução	*host*: execuções inteiras idênticas por `cmp`; 10.080 linhas
Idêntico por uma API de GPU	nenhum	*spirv*: os mesmos arquivos a partir de SPIR-V no Vulkan de CPU; 5.189 linhas
Idêntico numa GPU real	*gpu*: nenhum	*cuda*: nenhum

4 O que cada projeto encontrou

ww3-gpu: a referência é o código otimizado. A identidade byte a byte da DIA precisou de um fato que o texto-fonte não dá. O gfortran expande `x**11` como uma cadeia de multiplicações em `-O0` e como outra cadeia em `-O3`. As duas cadeias discordam em 643.427 de 1.000.000 entradas de precisão simples. Antes dessa descoberta, o diário de pesquisa explicava as diferenças restantes como arredondamento amplificado pelo WW3. Essa explicação estava errada. Por isso, a referência do porte é o código que o build do WW3 produz, lido como GIMPLE ou desmontagem, e não o texto Fortran.

O porte do ww3-gpu não tem testes unitários próprios. Só os testes de regressão do WW3, comparados pelo `test.comp`, julgam o porte.

chaceon: uma execução sem Fortran. Os programas Rust `ww3_grid`, `ww3_strt`, `ww3_prep` e `ww3_shel` rodam em sequência a partir das namelists. Os programas escrevem o `mod_def.ww3`, a forçante, as saídas em grade e em pontos, o arquivo de trajetória e os dois restarts. Os 96 arquivos são idênticos byte a byte à referência. As fronteiras dão 214 comparações idênticas em 34 casos, e os pós-processadores de trajetória dão 74. O `ww3_shel --threads N` e um build MPI dão os mesmos bytes.

Os arquivos continuam idênticos quando o `W3SRCE`, o `W3XYP3` e o `W3KTP3` são compilados para SPIR-V e rodam num driver Vulkan de CPU. A enumeração não alcança os 2^{64} pares de operandos `f32`. Nesse caso, uma prova em Lean enuncia o fato sobre a divisão que o porte usa. A prova não diz nada sobre a igualdade entre Rust e Fortran, porque o Fortran não tem semântica formal utilizável.

Por que o ww3-gpu ainda não tem prova, e o que vem depois. Nesta primeira fase, o ww3-gpu não adotou assistente de prova (Hoffmann 2026f). Há quatro motivos:

1. Nenhum assistente de prova lê Fortran ou C++. Uma prova trata de um modelo que uma pessoa escreve. A prova não diz nada sobre os binários do gfortran e do nvcc enquanto ninguém mostrar que o modelo concorda com esses binários, e essa concordância é a questão em aberto.
2. A igualdade de bits é uma propriedade de dois binários, compilados com flags dadas. A igualdade de bits não é uma propriedade de dois textos-fonte.
3. Para código IEEE-754 sem desvios, a igualdade em todas as entradas é decidível. Consultas SMT na teoria `QF_FP` e varreduras sobre as 2^{32} entradas `float32` respondem a pergunta em segundos ou minutos. Uma prova interativa leva meses para cada rotina.
4. Agentes precisam escrever cada prova, e uma pessoa precisa revisar cada prova. A prova é um segundo artefato para cada rotina.

Sem prova, o porte da DIA tem a evidência mais forte abaixo de uma prova. O porte é idêntico byte a byte nas 125 entradas da matriz de regressão do WW3 que selecionam a DIA (100 seriais, 25 OpenMP). A seção 1 do kernel também passou por uma varredura das 2^{32} entradas `float32`, com 0 diferenças (Hoffmann 2026f). Por isso, os testes do próprio modelo validam o porte da DIA em toda entrada que chama o porte. Essa validação não é uma prova formal, e este texto não chama a validação de prova.

Uma primeira camada formal já existe ao lado dos portes. O pacote Lean 4 `ocean-waves` (Hoffmann 2026b) enuncia a equação de balanço de ação discretizada e prova estas propriedades, sem `sorry` e sem `axiom`:

- $\sigma^2(k)$ cresce estritamente com k;
- $c/2 \le c_g \le c$;
- a condição CFL de profundidade mantém todo número de Courant em $[0, 1]$;

- a propagação upwind e uma troca de quadrupletos no formato da DIA conservam a ação;
- o passo semi-implícito do termo-fonte mantém a ação positiva para todo passo de tempo.

O pacote usa aritmética `Rat` exata num modelo reduzido: uma dimensão espacial, profundidade fixa e um modelo de Padé para tanh. Por isso, o pacote também não diz nada sobre bits. O Agda é o próximo passo. O Agda não tem tipo de ponto flutuante de 32 bits, então o Agda não consegue provar um fato sobre bits. O papel do Agda é uma semântica de referência total e executável de cada termo-fonte. Cada índice de tabela terá um tipo `Fin`, e toda leitura fica dentro dos limites para toda grade, não só para uma grade. As varreduras, as consultas ao solver e os testes de regressão do WW3 continuam a decidir os bits.

5 Desafios: os testes de regressão são grandes

Os testes de regressão do WW3 são o único juiz do porte Kokkos, e esses testes são grandes. A Tabela 4 dá o tamanho da matriz no commit `80644ba9` do fork, contado em 2026-10-18.

Tabela 4: Tamanho da matriz de regressão do WW3. Fonte: ww3-gpu `regtests/README.md` em `58167d7`, contagens de `just ww3-gpu-matrix-list`. A contagem de todas as entradas é o número de linhas que o `matrix.base` escreve, e uma seção de máquina consegue desligar algumas dessas linhas.

Conjunto de entradas do `matrix.base`	Entradas	Fração
Todas as entradas	758, em 60 testes	100 %
Entradas com a DIA (`NL1`)	310	41 %
Entradas com a física ST4 (`ST4`)	136	18 %
Entradas que o preset residente consegue selecionar	7 (todas seriais)	0,9 %
Regtests selecionados do ww3-gpu	5, e o par `ww3_ts1`	0,7 %

Uma execução das 310 entradas `NL1` leva dias. O conjunto selecionado leva de uma a três horas numa estação de trabalho (Intel i9-14900, 32 threads de hardware); esse tempo é uma estimativa, e não uma medida ⚠. Por isso, uma execução da matriz completa não consegue acompanhar cada commit. O ww3-gpu roda o conjunto selecionado como proteção e adia a matriz completa para uma fase de validação. Até essa fase, um argumento estático sustenta a paridade de bits: o GIMPLE em `-O3` ou a desmontagem dos dois compiladores.

6 O que este texto não afirma

1. Nenhum dos dois projetos rodou o modelo inteiro numa GPU real. O PTX do chaceon nunca rodou. O ww3-gpu rodou um kernel de benchmark da DIA numa RTX 4090 (Hoffmann 2026d).
2. Este texto não afirma ganho de velocidade. As regras do ww3-gpu marcam os tempos registrados (Hoffmann 2026e) como “regtest pending”, e o chaceon não tem tempos medidos.
3. Numa GPU, a identidade de bits também exige que o `expf` e o `logf` do dispositivo devolvam os bits da glibc. O ww3-gpu manda os kernels para cópias de dispositivo das funções da glibc 2.44. Essas cópias são iguais às funções do host em toda entrada conferida. Nenhuma execução em dispositivo comparou as cópias ainda. Até uma execução comparar, uma tolerância fixa julga uma execução em GPU.
4. As duas referências são linhas diferentes do WW3 (NOAA-EMC e MetOcean). Por isso, a comparação entre os dois portes é por nome de rotina, e não por texto-fonte. Os dois registros contêm 85 rotinas.

7 Próximos passos

Os autores planejam quatro passos:

1. Retomar o desenvolvimento do ww3-gpu e do chaceon numa NVIDIA RTX 4090.
2. Rodar os testes de regressão selecionados numa NVIDIA H100 da UFSC.
3. Fixar todos os runtimes no WW3 7.14.
4. Publicar a versão 1.0.0 do ww3-gpu e do chaceon antes de fevereiro de 2027.

Esses passos são planos, e não resultados ⚠. O primeiro autor supervisiona cada execução em GPU, como as regras do ww3-gpu exigem.

8 Uso de IA

Agentes de programação Claude (Anthropic) produziram o código C++/Kokkos e Rust, os shims, os scripts de comparação, as investigações e rascunhos deste texto. Os agentes trabalharam sob a direção do primeiro autor, nos dois repositórios. Pedro Cruz (GTA, UFRJ) é o orientador e Pedro Veras Guimarães (LabECO, UFSC) é o coorientador do primeiro autor.

A maior parte dos commits veio do Claude Opus. Na experiência do primeiro autor, o Claude Sonnet também portou uma quantidade grande de código sem erros grosseiros ⚠. Um exemplo é a divisão do `W3SDS4` em partes que rodam um ponto de mar por faixa vetorial (commit `9cef132` do ww3-gpu, 841 linhas acrescentadas). Os arquivos de saída desse commit foram idênticos byte a byte ao Fortran do mesmo executável nos casos `smoke` e `full`.

Cada repositório publica as regras, as skills e os hooks dos agentes no arquivo `AGENTS.md` e no diretório `.claude/`. Cada número deste texto vem de um arquivo de um repositório, no commit que o texto nomeia. Nada foi executado para este texto. Cada afirmação foi conferida em 2026-11-11 contra a fonte e o commit que ela nomeia; um ⚠ marca uma afirmação que não foi conferida numa fonte primária. A física do WW3 é usada sem mudança; WAVEWATCH III é marca registrada da NOAA.

Aviso. Hoje, os dois repositórios estão configurados para o Claude Code. Os autores vão continuar o trabalho para qualquer agente de programação conseguir usar o ww3-gpu e o chaceon. Esse trabalho inclui agentes que rodam modelos locais com o Ollama. O trabalho não está completo, e este texto não afirma resultados para outros agentes nem para modelos locais.

Referências

Hoffmann, Matheus. 2026a. "chaceon: a Rust port of WAVEWATCH III". https://github.com/h0ffmann/chaceon.

———. 2026b. "Ocean waves: the physics of a spectral wave model in Lean 4". https://github.com/h0ffmann/proof/tree/b863c15/ocean-waves.

———. 2026c. "WW3 GPU Lab: agentic coding and agentic research on a bit-for-bit GPU port of WAVEWATCH III". https://doi.org/10.5281/zenodo.23221351.

———. 2026d. "ww3-dia-gpu-v2: the whole W3SNL1, inside WAVEWATCH III, bit for bit". https://github.com/h0ffmann/ww3-dia-gpu-v2/tree/32aa2ef.

———. 2026e. "A bit-identical Kokkos runtime for WAVEWATCH III: CPU results and five speed-ups". ww3-gpu, `docs/NOAA_PROGRESS_RECORD_202610.md`, commit 58167d7. https://github.com/h0ffmann/ww3-gpu/blob/58167d7/docs/NOAA_PROGRESS_RECORD_202610.md.

———. 2026f. "ADR-0001: no proof language for the GPU port; SMT queries and exhaustive sweeps instead". ww3-gpu, `docs/ADRs/ADR-0001-proof-language.md`, commit 58167d7. https://github.com/h0ffmann/ww3-gpu/blob/58167d7/docs/ADRs/ADR-0001-proof-language.md.

Koldunov, Nikolay V., Suvarchal K. Cheedela, Sergey Danilov, Dmitry Sidorenko, Sebastian Beyer, e Thomas Jung. 2026. "An Ocean Model Ported by a Large Language Model: Experience and Lessons from FESOM2 (Fortran to C to C++/Kokkos)". arXiv:2606.11356 [physics.ao-ph]. https://doi.org/10.48550/arXiv.2606.11356.

LabECO/UFSC. 2025. "ReNOMO: Rede Nacional de Observação e Monitoramento Oceânico". https://labeco.ufsc.br/renomo/.

NOAA Environmental Modeling Center. 2026. "WAVEWATCH IV". https://github.com/NOAA-EMC/WW4.

Tolman, Hendrik L. 2025. "The WAVEWATCH III Software Modernization Project: Phase I report". Office Note 525. College Park, MD: NOAA/NWS/NCEP. https://doi.org/10.25923/h7j3-1h25.

Universidade Federal de Santa Catarina. 2026. "Ondas & Ventos". https://ondaseventos.ufsc.br/.